Machine Learning 1 · 2026-2027

Prédire la qualité d'un vin.

Une classification à partir de 11 mesures chimiques : les choix faits à chaque étape, et pourquoi.

Zakaria Hentabli

Professeur : Ahmed-Ghazi Blaiech

Le problème

Deux vins, deux jeux de données.

Chaque vin a une note de 0 à 10. Il est **bon** si sa note est de 7 ou plus.

Vin rouge

1 599

vins

13,5 %

de bons vins

Vin blanc

4 898

vins

20,8 %

de bons vins

Pourquoi deux modèles séparés ?

Le blanc a beaucoup plus de sucre et de soufre. Et ce qui rend un rouge bon n'est pas ce qui rend un blanc bon.

Vérifié : un modèle unique fait moins bien sur chaque vin.

Modèle unique	Deux modèles
0,860 · 0,815	0,872 · 0,822

Score AUC, rouge · blanc

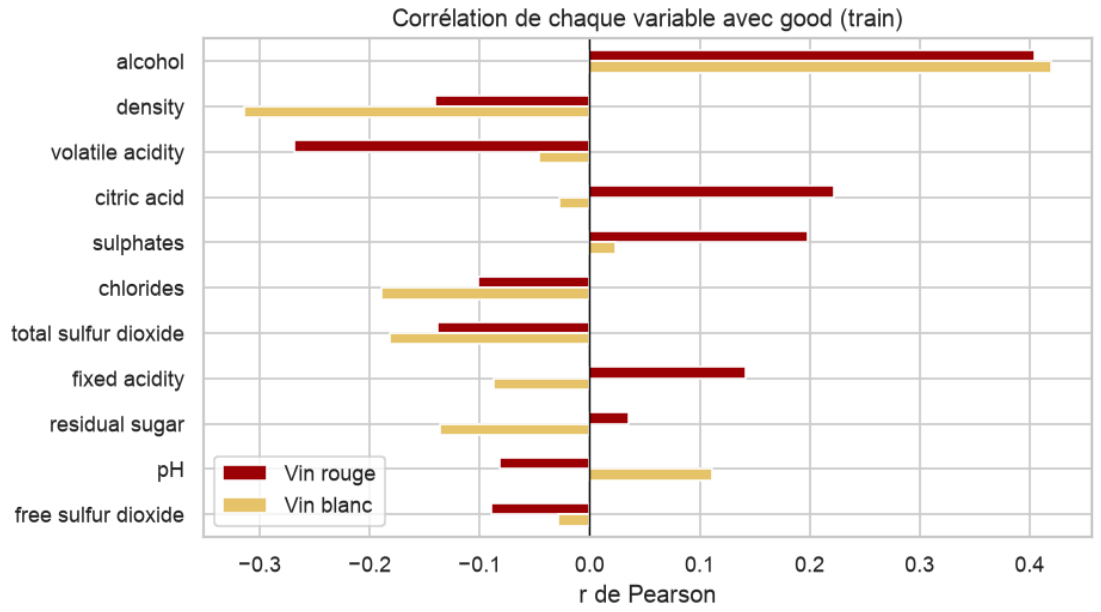
Étape 1 · Préparer

Nettoyer sans rien inventer.

Retirer les doublons	**1 177** copies (15 à 19 % des lignes). Une copie dans le test, c'est une réponse donnée d'avance.
Aucune valeur manquante	Rien à remplacer, donc rien à inventer.
Garder les valeurs extrêmes	Ce sont de vrais vins, pas des erreurs. On les adoucit (logarithme) au lieu de les supprimer.
Plus de poids aux bons vins	Ils sont rares : sans ça, le modèle apprend à toujours répondre « mauvais ».
20 % mis de côté	Gardés pour le **test final**, utilisé une seule fois. Toutes les décisions se prennent sur les 80 % restants.

Étape 2 · Explorer

L'alcool compte le plus.



Lien de chaque mesure avec « bon vin »

0,4

lien de l'alcool avec la qualité, pour les deux vins. Le plus fort, mais modéré : aucune mesure ne suffit seule.

Rouge ≠ blanc

Rouge : l'acidité volatile (goût de vinaigre) fait baisser la qualité. Blanc : c'est la densité.

Moins de variables ?

3 méthodes de sélection testées : **même score** avec moins de mesures.

Étape 3 · Évaluer

Choisir la bonne mesure.

86,5 %

de bonnes réponses pour un modèle qui dit **toujours « mauvais »**.

Il ne trouve **aucun** bon vin. Le taux de bonnes réponses est trompeur.

L'AUC

Le modèle sait-il placer les bons vins devant les mauvais ? **0,5** = hasard, **1** = parfait.

Le F1

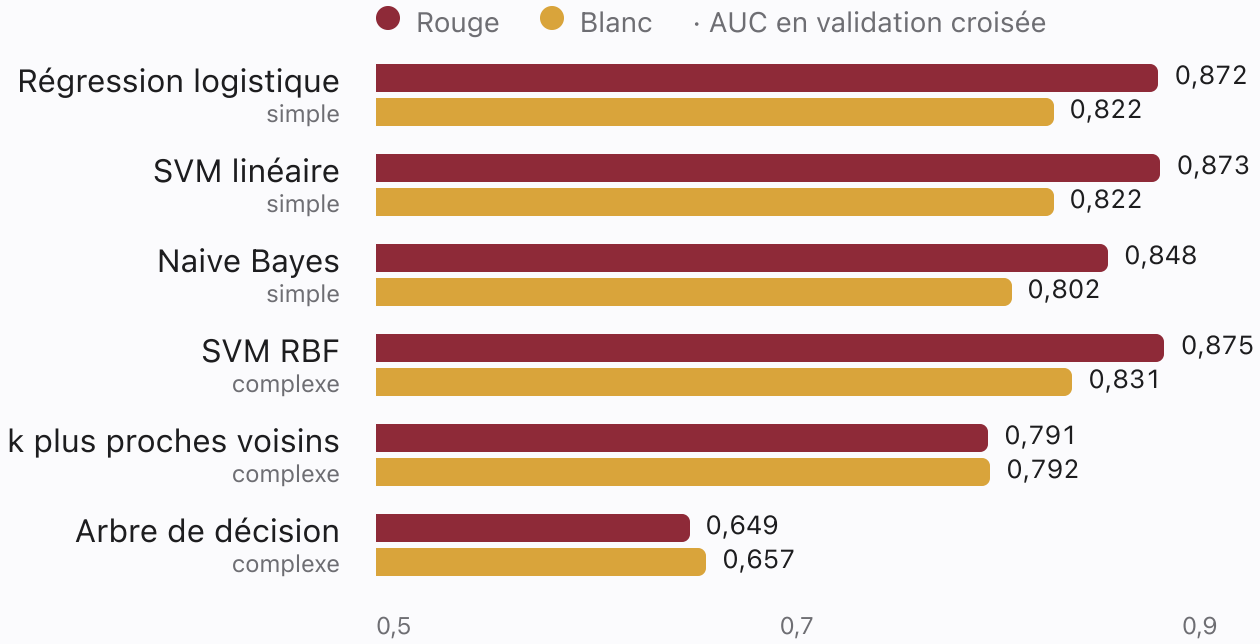
Trouver les bons vins, sans trop de fausses alertes.

La validation croisée

Les 80 % sont coupés en 5 parts : on apprend sur 4, on note sur la 5e, cinq fois. On obtient un score **et** sa marge d'erreur.

Étape 4 · Modéliser

Simple ou complexe ? Match nul.



Les simples font aussi bien

Le meilleur modèle complexe (SVM RBF) gagne au plus **0,01** : moins que la marge d'erreur.

L'arbre apprend par cœur

Sans limite : **1,00** sur les vins vus, **0,65** sur des vins nouveaux.

Étape 5 · Régler et choisir

Le réglage aide les faibles. Le plus simple gagne.

Arbre de décision · rouge

0,64 → 0,82

k plus proches voisins · rouge

0,81 → 0,87

Modèles simples

≈ 0 de gain

Tester les réglages	Grid Search et Random Search, 64 essais chacun : même score.
Ne pas se noter soi-même	Chaque réglage est noté sur des données qui n'ont pas servi à le choisir.
Choisir le plus simple	À score égal : la **régression logistique**, pour le rouge et le blanc. Rapide et lisible.

Étape 6 · Test final

Le test confirme les estimations.

Vin rouge

0,89

AUC sur le test (estimé avant : 0,87)

24/37

bons vins trouvés

Vin blanc

0,84

AUC sur le test (estimé avant : 0,82)

129/165

bons vins trouvés

Une seule fois

Modèle et réglages fixés **avant** d'ouvrir le test. Scores proches de l'estimation : la méthode était fiable.

Concrètement (rouge)

2 bons vins sur 3 sont trouvés. 1 vin annoncé bon sur 2 l'est vraiment.

Prudence

Seulement 37 bons vins rouges dans le test : marge d'environ **± 0,03**.

Conclusion

Ce qu'il faut retenir.

01

La qualité dépend surtout de **l'alcool**, et d'effets simples.

02

La **régression logistique** suffit : **0,89** (rouge) et **0,84** (blanc).

03

Une **méthode honnête**, test utilisé une fois, rend ces scores crédibles.

Limites et piste

Un vin noté 6 et un vin noté 7 se ressemblent beaucoup. Le test rouge est petit.
Piste : forêts aléatoires et boosting, hors du cadre du sujet.

Bonus

✓ Docker ✓ GitHub
✕ Tests unitaires ✕ Application web